Lab 00 — Answer key

Every answer follows the same pattern: the answer itself, the mechanism behind it, a nuance or pitfall, and an example you can verify at the terminal.

Question 1 — A Linux binary runs everywhere… except on Windows

Answer. The binary asks nothing of "Ubuntu" or "Alpine". It asks everything of the **kernel**, through system calls (`open`, `read`, `fork`…), and those calls are identical on every Linux machine. The distribution merely supplies the userland around them. The Windows kernel exposes a different, incompatible set of calls, so the binary has no one to talk to there.

Why. The interface between kernel and programs is stable and standardized — Linux famously refuses to break it. Everything that makes distributions different (package manager, library versions, configuration) lives *above* that interface.

Nuance. "Identical" assumes the required shared libraries are present — the `libc`, for instance, differs between Debian and Alpine, and you will run into that in lab 05. Also, WSL does not "translate" anything: WSL 2 runs a **real** Linux kernel inside a VM.

Example.

```
uname -r          # 6.6.87.2-microsoft-standard-WSL2: a real Linux kernel, built by Microsoft
uname -m          # x86_64: the architecture — the other compatibility requirement
```

Question 2 — The orphan adopted by PID 1

Answer. The shell — the parent of that `sleep` — died along with the terminal. The kernel never leaves a process parentless: PID 1 (`systemd`) **adopts** the orphan, which is why the PPID now reads `1`. The `sleep` itself keeps running as if nothing happened.

Why. A parent has a specific duty: when a child dies, the parent collects its exit code (until then, the child lingers as a "zombie"). So the system always needs a guardian of last resort — one of PID 1's jobs.

Nuance. This is exactly why the PID 1 *of a container* is a serious topic (lab 03): there, your application inherits that guardian role without knowing it. A PID 1 that never reaps its zombies, or has no fallback parent behind it, changes how the container behaves.

Example.

```
sleep 300 &
ps -o pid,ppid,cmd | grep [s]leep    #  2419  2363  sleep 300  (PPID = your bash)
# close the terminal, open a new one:
ps -ef | grep [s]leep                #  ubuntu  2419  1  ...  sleep 300  (PPID = 1)
```

Question 3 — `Permission denied`, code 126

Answer. The file is missing the execute bit `x`. Confirm it with `ls -l deploy.sh` — you will see `-rw-r--r--`, no `x` anywhere. Fix it with `chmod +x deploy.sh`. And the workaround: run `bash deploy.sh`. That launches `bash` (which *is* executable) and hands it the script as a mere argument to read.

Why. `./deploy.sh` asks the kernel to **execute this file**; the kernel checks the `x` bit and refuses. Code 126 is the shell's convention for "found, but not executable" — as opposed to 127, "not found".

Nuance. A file created by an editor or downloaded from the web starts out as `rw-`: the right to execute is something you grant deliberately. In a Dockerfile (lab 04), a `COPY` of a script followed by `RUN ./script.sh` fails in exactly the same way if the `x` bit was missing in your Git repository.

Example.

```
printf '#!/bin/bash\necho hello\n' > deploy.sh
./deploy.sh ; echo $?      # bash: ./deploy.sh: Permission denied ; 126
chmod +x deploy.sh
./deploy.sh                # hello
```

Question 4 — Shell variable vs environment variable

Answer. Line 1 prints `1:` (nothing). Line 2 prints `2: hello`. Before the `export`, `MSG` exists only inside **the current shell**; the `bash -c` child starts with the inherited environment, and `MSG` isn't in it. After `export`, `MSG` joins the environment, and every child gets a copy.

Why. At creation, a process receives a **copy** of its parent's environment — never a reference. Inheritance flows one way and is frozen at launch time.

Nuance. The **single** quotes in `'echo 1: $MSG'` matter: they stop your own shell from substituting `$MSG` before the child even starts. With double quotes, both lines would print `hello` — but the parent would have done the substitution, not the child. One important consequence: you cannot change the environment of a process that is **already running**. That is why `podman run -e` fixes everything at startup (lab 08).

Example.

```
MSG=hello
env | grep MSG          # nothing
export MSG
env | grep MSG          # MSG=hello
```

Question 5 — Code 137

Answer. 137 = **128 + 9**: the process died from signal number 9, `SIGKILL`. It got no chance whatsoever to shut down cleanly. The code is famous because it is what a terminated container leaves behind — after a `docker kill`, after a `docker stop` whose grace period ran out, or after the kernel's **OOM killer** stepped in when memory ran short.

Why. The shell encodes death-by-signal as `128 + signal number`, to tell it apart from a voluntary `exit n`. And `SIGKILL` is never actually delivered to the process: the kernel simply erases it, running no cleanup code at all.

Nuance. Diagnosing a 137 therefore means finding out **who** sent the 9 — a human, an orchestrator, or the kernel itself. `dmesg | grep -i "out of memory"` settles the OOM case. You will run this diagnosis on real containers in lab 03.

Example.

```
bash -c 'kill -9 $$' ; echo $?    # 137  ($$ = the PID of the child bash itself)
sleep 300 & kill -9 $! ; wait $! ; echo $?   # 137 as well
```

Question 6 — Polite `kill`, brutal `kill -9`

Answer. Plain `kill` sends `SIGTERM`, which is a **request**: the process receives it, gets to run its shutdown code — flush buffers, finish transactions, close connections — and then exits. `kill -9` sends `SIGKILL`, which is never delivered at all: the kernel wipes the process out on the spot. A database then loses whatever hadn't reached the disk yet, and has to replay its journal on restart — or worse, repair corrupted files.

Why. This is precisely what `docker stop` does: it sends `SIGTERM` to the container's PID 1, waits out a grace period (10 seconds by default), then sends `SIGKILL` if the process hasn't complied. An application that ignores `SIGTERM` therefore **always** dies the hard way once the delay expires.

Nuance. `kill -9` has its place — for a stuck process that genuinely ignores `SIGTERM`. The right habit is to escalate: `kill`, wait, and only then `kill -9`. Never the other way around. Note that `SIGKILL` can be neither caught nor ignored, which makes it the one guaranteed remedy.

Example.

```
sleep 300 &
kill %1        # SIGTERM: the job reports "Terminated"
sleep 300 &
kill -9 %1     # SIGKILL: the job reports "Killed"
```

Question 7 — Reading `-rw-r----- root shadow`

Answer. The nine bits split into three triplets: owner `rw-`, group `r--`, others `---`. Your user is neither `root` (the owner) nor a member of the `shadow` group, so the "others" triplet applies — and it grants nothing. Hence the refusal. `sudo cat` runs `cat` with UID 0, and the kernel skips permission checks for root entirely. Without `sudo`, only root and members of the `shadow` group (read-only) can open the file.

Why. On every `open`, the kernel compares the calling **process's** UID/GID against the file's bits: owner first, then group, then others. The first triplet that matches is the only one applied.

Nuance. `/etc/shadow` holds the password hashes — it is the textbook example. And the "first matching triplet" rule can surprise you: a file with permissions `----rw-rw-` would be unreadable... by its own owner.

Example.

```
id                      # uid=1000(ubuntu) ...: not root, not in group shadow
cat /etc/shadow         # Permission denied, code 1
sudo head -n 1 /etc/shadow   # root:*:20501:0:99999:7:::
```

Question 8 — Two streams, two files

Answer. The screen shows **nothing**. `result.txt` contains the success line (`/etc/hostname`); `errors.txt` contains `ls: cannot access '/no-such-path': No such file or directory`. With `> result.txt 2>&1`, both lines would land in `result.txt`, and `errors.txt` would not be created.

Why. `ls` writes results to **stdout** (stream 1) and complaints to **stderr** (stream 2). `>` redirects only stream 1, `2>` only stream 2. `2>&1` means "point stream 2 wherever stream 1 points *right now*".

Nuance. Order matters: `2>&1 > result.txt` would send the errors to the screen, because stream 2 gets attached to the old destination of stream 1, before the redirection. This separation of streams is what lets `podman logs` show you both a container's errors and its normal output (lab 03).

Example.

```
ls /etc/hostname /no-such-path > result.txt 2> errors.txt
cat result.txt      # /etc/hostname
cat errors.txt      # ls: cannot access '/no-such-path': No such file or directory
```

Question 9 — `127.0.0.1` vs `0.0.0.0`

Answer. Redis listens on `127.0.0.1:6379` — the *loopback* interface only — so it can be reached **only from the machine itself**. Java listens on `0.0.0.0:8080` — every interface — so it can be reached from the network. From another machine, only the Java service answers.

Why. The listening address acts as a filter: the kernel only hands the process connections that arrived on that address. `0.0.0.0` means "every address this machine has".

Nuance. This is a security boundary of the first order: a database that listens locally simply cannot be attacked over the network. In lab 07 you will see that `podman run -p 8080:80` publishes on `0.0.0.0` by default, and that `-p 127.0.0.1:8080:80` deliberately narrows it. If you understand these two `ss` lines, you already understand `-p`.

Example.

```
python3 -m http.server 8080 --bind 127.0.0.1 &
ss -tlnp | grep 8080     # LISTEN ... 127.0.0.1:8080 ... ("python3",pid=...)
kill %1
```

Question 10 — Port 80 refused

Answer. Ports **below 1024** — the so-called *privileged* ports — can only be opened by root (UID 0). Your process runs as UID 1000, so the kernel refuses its `bind` on port 80; 8080 sits above the threshold and is free for anyone. Historically, the rule guaranteed that on a shared machine, no ordinary user could impersonate an "official" service (port 25, port 80…). The consequence today: rootless Podman, being an ordinary process under your UID, cannot publish `-p 80:80` — so you publish `-p 8080:80` instead.

Why. The kernel performs the check during the `bind` system call, based on the effective UID (strictly speaking, on a *capability* that root holds).

Nuance. The threshold can be tuned (`sysctl net.ipv4.ip_unprivileged_port_start`), and real production web servers sit behind a load balancer that owns port 80 for them. Podman's version of this error (`pasta failed ... Permission denied`) returns in lab 07.

Example.

```
python3 -m http.server 80
# PermissionError: [Errno 13] Permission denied
python3 -m http.server 8080 &   # works
kill %1
```

Question 11 — `/proc`, the directory that isn't there

Answer. `/proc` is a **virtual filesystem** (type `proc`) mounted on `/proc`. Its contents exist on no disk: the kernel fabricates every read on the fly from its internal state. The numbered directories are the living processes; the other files describe the system. Typical uses: `/proc/<pid>/environ` (a process's actual environment), `/proc/meminfo` (memory), `/proc/self/uid_map` (the UID mappings — your proof of rootless mode in lab 01).

Why. "Everything is a file": exposing kernel state as files means `cat`, `grep` and `ls` become administration tools, with no special API needed. `ps` is nothing more than a front-end for `/proc`.

Nuance. This is also why every container gets **its own** `/proc` mounted at creation — otherwise it would see all the host's processes. When `ps` seems to lie inside a container, it is the container's isolated `/proc` talking, not processes disappearing.

Example.

```
findmnt -t proc          # /proc  proc  proc  rw,relatime
df -h /proc 2>/dev/null  # no disk behind it
tr '\0' '\n' < /proc/self/environ | head -3   # the environment... of this very cat
```

Question 12 — `command not found`, code 127

Answer. The shell walked through every directory in `PATH`, in order, looking for an executable named `mytool`. `~/tools` is not on that list, so the search failed with code 127. Immediate workaround: use an explicit path, `~/tools/mytool`. Permanent fixes: (1) add the directory to the PATH in `~/.bashrc` (`export PATH="$HOME/tools:$PATH"`), or (2) copy or symlink the tool into a directory already on the list, such as `~/.local/bin` or `/usr/local/bin`.

Why. The PATH is the only way the shell resolves "bare" command names. The current directory is left off the list on purpose: a booby-trapped `ls` dropped into `/tmp` must not run just because you happened to `cd /tmp`.

Nuance. Keep 127 (not found) and 126 (found but not executable) apart — they are two different diagnoses. Inside a container, the error `exec: "mytool": executable file not found in $PATH` has exactly the same cause: the image's PATH (lab 04).

Example.

```
mkdir -p ~/tools && printf '#!/bin/bash\necho ok\n' > ~/tools/mytool && chmod +x ~/tools/mytool
mytool                   # command not found ; echo $? → 127
export PATH="$HOME/tools:$PATH"
mytool                   # ok
```

Question 13 — Why 12-factor loves the environment

Answer. Because the environment belongs to the **process**, not to the machine: it is set at launch, inherited automatically, and disappears with the process. For disposable, restartable applications, that yields configuration which (1) can be injected from the outside without touching the code or the shipped files, (2) can differ per instance — two processes side by side, two configurations, and (3) leaves no residue: restart with new values, and there is nothing to clean up.

Why. Parent-to-child inheritance does all the work. Whoever launches the process — the shell, systemd, later the container engine — prepares the dictionary; the application just reads it. The same artifact, JAR or image, travels unchanged from one environment to the next; only the set of variables changes.

Nuance. The inheritance being frozen is also its limit: changing a variable means **restarting** the process. For a container — disposable by design — that costs nothing; for anyone hoping to reconfigure a live process, it is a hard no. Secrets, meanwhile, will deserve better than variables readable in `/proc/<pid>/environ` (lab 08).

Example.

```
SERVER_PORT=9090 java -jar app.jar   # same JAR, different port — nothing was modified
# which later becomes, word for word:
# podman run -e SERVER_PORT=9090 my-api
```